

Design and Development of an Intelligent Core Java-Based Student Academic Management System for Performance Monitoring and Decision Support

A CAPSTONE PROJECT REPORT

Submitted in the partial fulfilment for the Course of

CSA0927 PROGRAMMING IN JAVA

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

B TECH-AI & DS and IT

Submitted by

D. Dhanusha (192424189)

Priyadharshini K (192421236)

Under the Supervision of

Dr. S. Pushpalatha



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

August 2026



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105



DECLARATION

We declare that the report entitled “**Design and Development of an Intelligent Core Java-Based Student Academic Management System for Performance Monitoring and Decision Support**” is a unique and original work submitted for the degree of Bachelor of Technology. This work is a record of the capstone project for the course **CSA0927 Programming in Java**, carried out under the guidance of **Dr. S. Pushpalatha**, and has not been submitted for the award of any other degree or diploma in this or any other institution of higher learning. The project team members are D. Dhanusha (192424189), Priyadharshini K (192421236).

D. Dhanusha (192424189)

Priyadharshini K (192421236)

Date:

Place:



**SIMATS ENGINEERING,
SAVEETHA INSTITUTE OF MEDICAL AND
TECHNICAL SCIENCE, CHENNAI- 602105**



BONAFIDE CERTIFICATE

We declare that the report entitled “**Design and Development of an Intelligent Core Java-Based Student Academic Management System for Performance Monitoring and Decision Support**” is a unique and original work submitted for the degree of Bachelor of Technology. This work is a record of the capstone project for the course **CSA0927 Programming in Java**, carried out under the guidance of **Dr. S. Pushpalatha**, and has not been submitted for the award of any other degree or diploma in this or any other institution of higher learning. The project team members are D. Dhanusha (192424189), Priyadharshini K (192421236).

SIGNATURE

Dr. Sri Ramya

Program Director

Department of CSE

Saveetha School of Engineering

SIMATS

SIGNATURE

Dr. S. Pushpalatha

Professor

Department of CSE

Saveetha School of Engineering

SIMATS

Submitted for the Project work Viva-Voce held on _____ .

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, **Dr. N.M. Veeraiyan**, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, **Dr. Deepak Nallaswamy Veeraiyan**, and our Vice-Chancellor, **Dr. S. Suresh Kumar**, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, **Dr. B. Ramesh** for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department, Dr. Sri Ramya for her continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide, **Dr. S. Pushpalatha** for her creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature with Student Name

INDEX

CHAPTER NO	TOPICS	PAGE NO
1	Introduction 1.1 Background Information 1.2 Project Objectives 1.3 Significance of the Project 1.4 Scope of the Project 1.5 Methodology Overview	7-8
2	Problem Identification and Analysis 2.1 Description of the Problem 2.2 Evidence of the Problem 2.3 Stakeholders and Affected parties 2.4 Supporting Data and Research	9-10
3	Solution Design and Implementation 3.1 Development and Design Process 3.2 Tools and Technologies Used 3.3 Overview of the Solution 3.4 Engineering Standards Applied 3.5 Justification of the Solution	11-12
4	Result and Recommendation 4.1 Evaluation of Result 4.2 Challenges Encountered 4.3 Possible Improvement to the Solution 4.4 Recommendation	13
5	Reflection on learning and Personal Development 5.1 Key Learning outcomes 5.2 Challenges Overcomes and Lessons learned 5.3 Application of Engineering Standards 5.4 Insights into the Industry 5.5 Conclusion of Personal Development	14
6	Conclusion	15
	References	16
	Appendices	17

ABSTRACT

The **Design and Development of an Intelligent Core Java-Based Student Academic Management System for Performance Monitoring and Decision Support** is developed to provide an efficient and organized solution for managing student academic information and supporting faculty in monitoring student performance. Traditional academic record management often involves manual data entry, separate records, and time-consuming analysis, which can make it difficult for faculty to identify students who require academic support. The proposed system addresses these issues through a simple and structured Core Java-based application.

The system is divided into **two major modules**. **Module 1: Student Academic Management System** focuses on maintaining student academic information such as student details, attendance, assignment scores, quiz scores, internal marks, and study hours. The system provides functions for adding, updating, searching, and managing student records in an organized manner.

Module 2: Performance Monitoring and Decision Support analyzes the collected academic information and categorizes students based on their overall performance. Students can be classified into categories such as **Excellent, Good, Average, and Poor**. The system also monitors attendance and generates alerts when a student's attendance falls below the defined threshold. These features help faculty members identify performance gaps and take appropriate academic-support actions.

The system uses **Core Java concepts**, including classes and objects, methods, arrays or collections, conditional statements, exception handling, and data processing. The application is designed to demonstrate how programming concepts can be applied to solve a practical academic management problem. Overall, the proposed system reduces manual effort, improves academic record organization, provides faster performance analysis, and supports faculty in making informed academic decisions.

Keywords: Core Java, Student Academic Management, Performance Monitoring, Decision Support, Academic Data Management

Keywords: Student Performance Analysis, Attendance Monitoring, Academic Records, Performance Prediction, Faculty Support

CHAPTER 1

Introduction

Education institutions handle large amounts of student academic information such as student details, attendance, assignment scores, quiz scores, internal marks, and study hours, but managing these records manually can be time-consuming, error-prone, and difficult to analyze. The **Design and Development of an Intelligent Core Java-Based Student Academic Management System for Performance Monitoring and Decision Support** is developed to provide an organized and efficient solution for managing student academic records and supporting faculty in monitoring performance. The system consists of two major modules: **Module 1: Student Academic Management System**, which manages and maintains student academic information, and **Module 2: Performance Monitoring and Decision Support**, which analyzes academic data and categorizes students into performance levels such as Excellent, Good, Average, and Poor while generating attendance alerts for students requiring attention. The system applies Core Java concepts such as classes and objects, methods, collections, conditional statements, exception handling, and data processing to provide a simple and reliable academic management solution. Overall, the proposed system reduces manual effort, improves data organization, enables faster performance analysis, and helps faculty members make informed decisions to improve student academic outcomes.

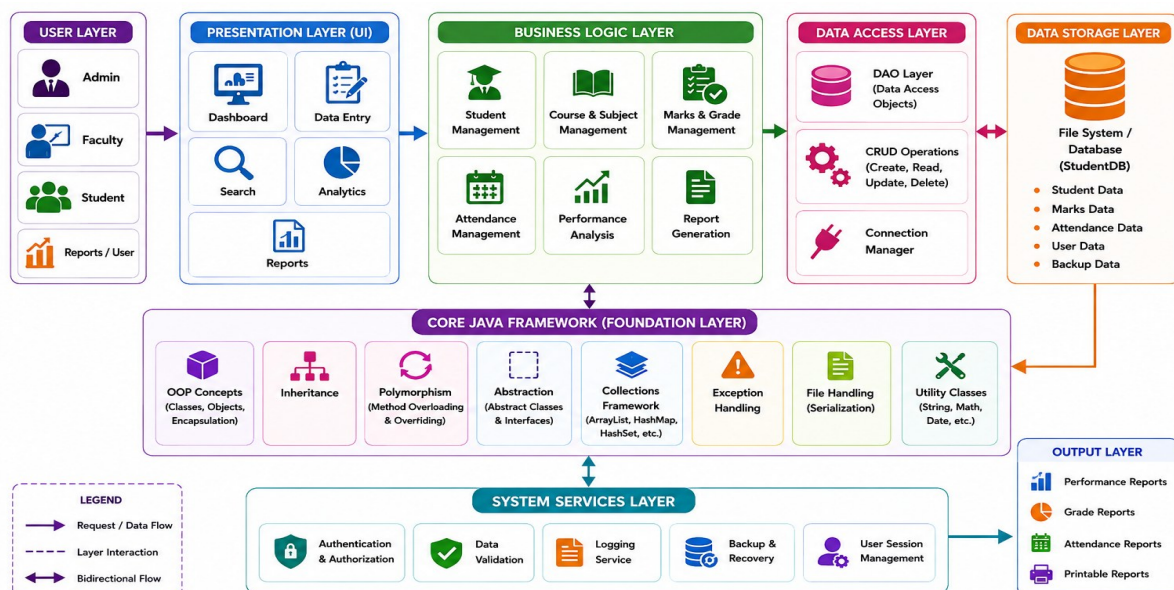


Fig 1: System Architecture

1.1 Background Information

Student academic records contain important information such as attendance, assignment marks, quiz scores, internal marks, and study hours. Managing all these details manually can take more time and may cause errors. It can also be difficult for teachers to quickly identify students who are performing well or need additional support. To solve this problem, the proposed Core Java-Based Student Academic Management System helps store and manage student records in an organized way. It also monitors student performance and provides useful information to teachers for making better academic decisions.

1.2 Project Objectives

The main objectives of the project are:

- To develop a Core Java-based system for managing student academic records.
- To store student details, attendance, assignment scores, quiz scores, internal marks, and study hours.
- To monitor and analyse the academic performance of students.
- To classify students into performance categories such as Excellent, Good, Average, and Poor.
- To identify students with low attendance or poor academic performance.
- To provide useful information to faculty members for better academic decision-making.
- To reduce manual work and improve the accuracy of academic record management.

1.3 Significance of the Project

The project is useful because it makes **student academic management easier and more organized**. It reduces the time and effort required to maintain student records manually. The system helps faculty members quickly view attendance and academic performance and identify students who may need additional support. It also provides performance categories and attendance alerts, which help teachers make better academic decisions. Overall, the system improves **accuracy, efficiency, and easy access to student academic information**.

1.4 Scope of the Project

- Manage **student personal and academic details**.
- Store **attendance, assignment, quiz, and internal marks**.

- Record **study hours** for performance analysis.
- Monitor overall **student academic performance**.
- Classify students as **Excellent, Good, Average, or Poor**.
- Generate **low-attendance and performance alerts**.
- Support faculty in making **better academic decisions**.

The scope of this project is to develop a **Core Java-based system** for managing and monitoring student academic information. The system covers student details, attendance, assignment scores, quiz scores, internal marks, and study hours.

It allows faculty members to maintain records, view student performance, identify low-performing students, and generate attendance alerts. The system also classifies students into different performance levels such as **Excellent, Good, Average, and Poor**.

The project mainly focuses on academic record management and performance-based decision support and can be further extended in the future with a database, web application, mobile application, or machine learning-based prediction features.

1.5 Methodology Overview

The project follows a simple **step-by-step development methodology**:

1. **Requirement Analysis** – Identify the academic management and performance monitoring needs.
2. **System Design** – Design the two modules and their functions.
3. **Data Collection** – Collect student details, attendance, marks, and study hours.
4. **Implementation** – Develop the system using **Core Java and OOP concepts**.
5. **Performance Analysis** – Calculate and classify student performance.
6. **Testing** – Test all functions and verify the accuracy of results.
7. **Evaluation** – Analyse the system and identify possible future improvements.

CHAPTER 2

Problem Identification and Analysis

The main problem is that **student academic information is often managed manually**, making it difficult to maintain accurate records and monitor performance regularly. Faculty members may need to check attendance, assignment marks, quiz scores, internal marks, and study hours separately, which takes time and can cause errors. It is also difficult to quickly identify students with **low attendance or poor academic performance**. Therefore, a Core Java-based system is needed to organize student records, monitor performance, and provide useful information for timely academic decisions.

2.1 Description of the Problem

Managing student academic records manually is time-consuming and may lead to errors. Faculty members may find it difficult to maintain attendance, marks, assignments, quiz scores, and study-hour records in one place. It is also difficult to quickly identify students with poor performance or low attendance. Therefore, a system is needed to organize academic data and help faculty monitor student performance effectively. In addition, the absence of a centralized system makes it difficult to obtain a clear overall view of a student's academic progress. Therefore, there is a need for a **Core Java-based Student Academic Management System** that can organize academic records, monitor performance, identify problem areas, generate alerts, and support faculty members in making timely academic decisions.

2.2 Evidence of the Problem

- Manual record maintenance requires more time.
- Student information may be stored in different files or registers.
- Data entry mistakes can occur frequently.
- Identifying low-performing students takes more effort.
- Low attendance may not be noticed immediately.
- Comparing the performance of many students is difficult.
- Lack of automatic alerts delays academic support.
- **Attendance Issues** – Low-attendance students may not receive timely alerts or academic guidance.

- **No Automatic Classification** – Manual records do not easily classify students into **Excellent, Good, Average, and Poor** categories.

2.3 Stakeholders and Affected Parties

- **Students** – Receive better monitoring and academic support. The system helps students understand their performance, attendance status, and areas where improvement is required.
- **Faculty Members** – Easily manage and analyze student records. Faculty can view marks, attendance, study hours, and performance categories to identify students who need additional guidance.
- **Department Staff** – Maintain organized academic information. The system reduces manual record maintenance and makes student information easier to access and update.
- **Academic Management** – Use performance information for decision-making. Department heads and academic coordinators can use summarized performance information to monitor student progress and plan suitable academic activities.
- **Institution** – Improves the efficiency of academic monitoring. The system reduces manual work, improves record organization, and supports the institution in maintaining effective student performance monitoring.

2.4 Supporting Data and Research

The system uses important academic parameters such as **attendance, assignment scores, quiz scores, internal marks, and study hours** to understand student performance. These parameters provide a basic view of a student's academic progress. Performance categories such as **Excellent, Good, Average, and Poor** can be used to identify students who need attention. Attendance monitoring also helps faculty detect students whose attendance falls below the required level. The proposed system combines these factors in one application to provide organized academic information and decision support.

- **Performance Classification** – Students can be grouped into **Excellent, Good, Average, and Poor** categories for easier monitoring.
- **Attendance Alert** – Students whose attendance falls below the defined threshold can be identified for further attention.
- **Internal Marks** – Helps evaluate overall academic performance before the final examination.

CHAPTER 3

Solution Design and Implementation

The proposed **Design and Development of an Intelligent Core Java-Based Student Academic Management System for Performance Monitoring and Decision Support** provides a structured solution for managing student academic information and analysing performance. The system is designed using **Core Java and Object-Oriented Programming concepts**. It consists of two major modules: **Module 1 – Student Academic Management System** and **Module 2 – Performance Monitoring and Decision Support**. The first module focuses on maintaining student academic records, while the second module processes these records to monitor performance and support faculty decision-making.

3.1 Development and Design Process

- **Requirement Analysis** – Identify the required student details, attendance, assignment marks, quiz scores, internal marks, and study hours.
- **System Design** – Design the system with two modules: Student Academic Management and Performance Monitoring & Decision Support.
- **Implementation** – Develop the system using **Core Java, OOP concepts, classes, objects, methods, and collections**.
- **Performance Analysis** – Process the academic data, calculate performance, classify students, and generate attendance alerts.
- **Testing and Evaluation** – Test the system with different student records, identify errors, correct them, and verify the final results.
- **Future Enhancement Planning** – Identify possible improvements such as database connectivity, GUI, web-based access, and advanced performance prediction.

3.2 Tools and Technologies Used

1. **Core Java** – Used as the main programming language to develop the application and implement the system logic.
2. **Object-Oriented Programming (OOP)** – Classes, objects, methods, encapsulation, and inheritance concepts are used to organize the system.
3. **Java Collections** – ArrayList and other collections can be used to store and manage multiple student records efficiently.

4. Java Exception Handling – Used to handle invalid inputs and runtime errors without stopping the entire application.
5. Java Compiler / IDE – Used to write, compile, execute, and debug the Java programs.
6. Data Processing – Conditional statements, loops, methods, and calculations are used to analyze attendance and academic marks.
7. Testing Data – Sample student academic records are used to test performance classification, attendance alerts, and other system functions.

3.3 Overview of the Solution

The proposed system is developed as a **Core Java-based Student Academic Management System** with two main modules:

1. Module 1 – Student Academic Management System

- Stores student personal and academic details.
- Manages attendance, assignment scores, quiz scores, internal marks, and study hours.
- Provides functions to add, update, search, and view student records.

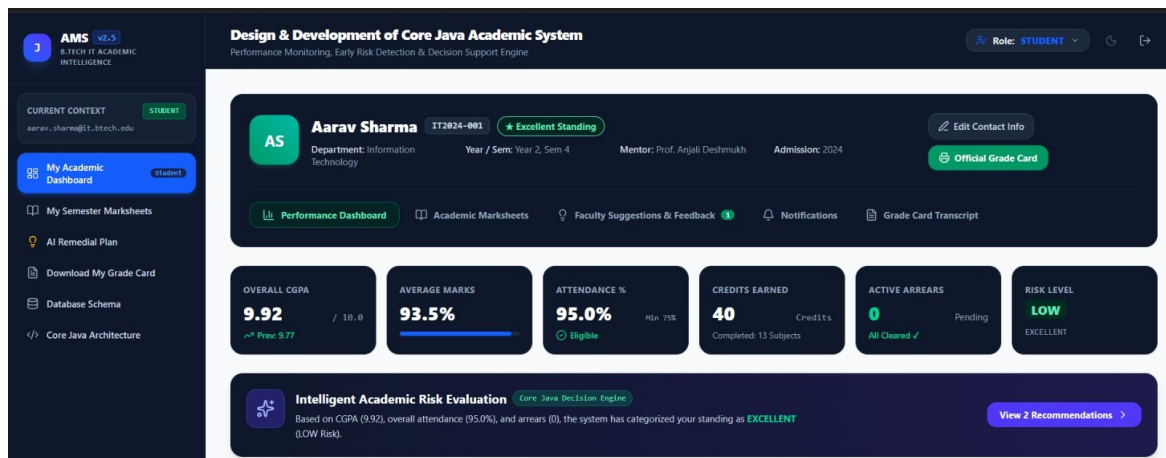


Fig 2: Dashboard

2. Module 2 – Performance Monitoring and Decision Support

- Analyses the academic information of students.
- Calculates overall performance based on the available academic parameters.

- Classifies students into **Excellent, Good, Average, and Poor** categories.
- Identifies students with low attendance or poor performance.
- Provides useful information to faculty for taking suitable academic decisions.

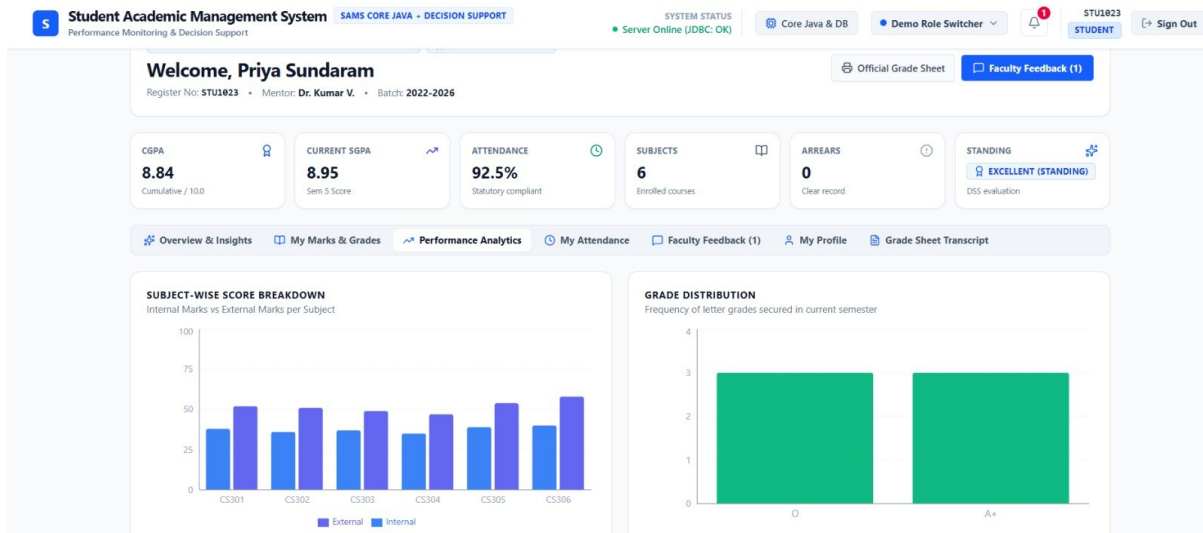


Fig 3: Performance Analysis

Overall workflow:

Student Data → Academic Record Management → Data Processing → Performance Analysis
→ Performance Classification → Alerts → Faculty Decision Support

3.4 Engineering Standards Applied

1. **Modular Design** – The system is divided into two modules, making development and maintenance easier.
2. **Object-Oriented Design** – Classes, objects, methods, and encapsulation are used to organize the application.
3. **Data Validation** – Student details, attendance, and marks are checked to reduce incorrect or invalid data.
4. **Exception Handling** – Java exception handling is used to manage errors and prevent sudden program termination.

3.5 Justification of the Solution

The proposed solution is suitable because it provides a simple, organized, and efficient way to manage student academic information. The system combines student record management with performance monitoring in a single application.

1. Reduces Manual Work – Reduces the time required to maintain student academic records manually.
2. Improves Data Organization – Keeps student details, attendance, marks, and study hours in an organized structure.
3. Easy Performance Monitoring – Helps faculty quickly understand the academic performance of each student.
4. Identifies Students Needing Support – Performance categories and attendance alerts help identify students who require additional academic guidance.
5. Supports Decision-Making – Provides useful academic information that helps faculty make timely decisions.

CHAPTER 4

Result and Recommendation

4.1 Evaluation of Result

The developed system was tested using sample student academic data. It successfully manages student details, attendance, assignment marks, quiz scores, internal marks, and study hours. The system also calculates student performance and classifies them into **Excellent, Good, Average, and Poor** categories. Low-attendance students can be identified easily through alerts. Overall, the system reduces manual work and helps faculty monitor student performance and make better academic decisions.

4.2 Challenges Encountered

During the development of the project, some challenges were faced. Managing multiple student academic records and maintaining accurate data was one of the main challenges. Designing the performance calculation and classification logic also required careful testing. Handling incorrect or missing inputs and debugging Java errors took additional time. Another challenge was ensuring that attendance alerts and performance categories worked correctly for different student records. These challenges were solved through repeated testing, debugging, and improving the program logic.

4.3 Possible Improvements to the Solution

The system can be improved by adding a **database** to store a large number of student records securely. A **Graphical User Interface (GUI)** can make the system easier for faculty to use. A dashboard can be added to display attendance and performance results clearly. The system can also include **automatic notifications** for low attendance and poor performance. In the future, **machine learning** can be integrated to predict student performance and identify students who may need early academic support.

4.4 Recommendation

It is recommended that the proposed **Student Academic Management System** be used to reduce manual academic record management and improve student performance monitoring. Faculty members should regularly update attendance, assignment marks, quiz scores, internal marks, and study hours to maintain accurate records. The system should be enhanced with a database and user-friendly interface for better usage. Regular testing and data backup should also be followed to maintain reliability. In the future, machine learning, automated notifications, and dashboards can be added to provide more advanced performance analysis and decision support.

CHAPTER 5

Reflection on Learning and Personal Development

5.1 Key Learning Outcomes

Through this project, I learned how to develop a **Core Java-based application** using classes, objects, methods, and basic OOP concepts. I gained practical knowledge in managing student

academic data, calculating performance, and identifying low-performing students. I also improved my skills in **problem-solving, coding, testing, and debugging**. The project helped me understand how software can be used to monitor academic performance and support better decision-making.

5.2 Challenges Overcome and Lessons Learned

During the project, I faced challenges in **writing the Java code, managing student data, and developing the performance calculation logic**. Some errors occurred while testing the program, which helped me improve my debugging skills. I also learned how to handle different student records and validate the input data. By solving these challenges, I improved my **problem-solving, programming, testing, and time-management skills**. The project taught me that careful planning and repeated testing are important for developing a reliable software system.

5.3 Application of Engineering Standards

During the project, basic software engineering standards were followed to make the system organized and reliable. The project used modular design, object-oriented programming, proper naming, code comments, data validation, exception handling, testing, and debugging. The system was divided into two modules to make development and maintenance easier. Different test cases were used to check the correctness of student records, performance calculations, and attendance alerts. These practices helped improve the quality, readability, reliability, and maintainability of the developed system.

5.4 Insights into the Industry

This project helped me understand how **software systems are used in the education industry** to manage student information and monitor academic performance. I learned that accurate data, automation, and easy-to-use systems are important for educational institutions. I also understood the importance of **data security, regular updates, testing, and user-friendly design** when developing real-world applications. The project gave me practical knowledge of how Java programming and software engineering concepts can be applied to solve real academic management problems.

5.5 Conclusion of Personal Development

This project helped me understand how **software systems are used in the education industry** to manage student information and monitor academic performance. I learned that accurate data, automation, and easy-to-use systems are important for educational institutions. I also understood the importance of **data security, regular updates, testing, and user-friendly design** when developing real-world applications. The project gave me practical knowledge of how Java programming and software engineering concepts can be applied to solve real academic management problems.

Conclusion

The **Design and Development of an Intelligent Core Java-Based Student Academic Management System for Performance Monitoring and Decision Support** provides a simple and organized solution for managing student academic information. The system consists of two main modules: **Student Academic Management System** and **Performance Monitoring and Decision Support**. It manages important information such as student details, attendance, assignment scores, quiz scores, internal marks, and study hours.

The system helps faculty members monitor student performance by analyzing the collected academic information and classifying students into **Excellent, Good, Average, and Poor** categories. It also identifies students with low attendance and provides alerts for further attention. The project demonstrates the practical use of **Core Java concepts**, including classes, objects, methods, collections, conditional statements, exception handling, testing, and debugging. It reduces manual work, improves data organization, and supports better academic decision-making.

The project successfully achieves its main objectives and provides a foundation for future development. The system can be improved by adding a **database, GUI, web application, dashboard, automated notifications, and machine-learning-based performance prediction**. Overall, this project provided practical knowledge in Java programming, problem-solving, software development, and academic data management while demonstrating how technology can be used to improve student performance monitoring in educational institutions.

References

1. Yarram, V. K., Parimi, S. K., Cherukuri, R., & Jegajothi, B. (2025, December). Intelligent Web based Student Performance Prediction using Machine Learning in Java. In *2025 4th International Conference on Applied Artificial Intelligence and Computing (ICAAIC)* (pp. 591-598). IEEE.
2. Rida, Z., Hadhoum, B., Mourad, E., & Mustapha, M. (2025). Dynamic adaptation in an intelligent multi-tutoring system: A multi-agent approach. *IEEE Access*, *13*, 124587-124601.
3. Latif, S., XianWen, F., & Wang, L. L. (2021). Intelligent decision support system approach for predicting the performance of students based on three-level machine learning technique. *Journal of Intelligent Systems*, *30*(1), 739-749.
4. Sagne, A., Sonti, P., Sahu, S., & Bhatt, N. 29. Advanced Student Management System: A Scalable and Intelligent Architecture for Academic Administration. *Intelligent, Sustainable, Social, and Secure Computing*, 259.
5. Kapoor, S. (2025). A Java-based system for collecting and analyzing college feedback. *Journal of Integrated Engineering Sciences*, *1*(1), 12-18.
6. Peng, L., & Yu, L. (2025, December). Validation of an AI-Driven Model for Enhancing Debugging Capabilities in Java Web Development Teaching. In *Proceedings of the 2025 3rd International Conference on Information Education and Artificial Intelligence* (pp. 1193-1198).
7. Eze Blessing, E., & Olaiya, O. O. (2025). Design and simulation of computerized student information tracking system. *International Journal of Scientific and Technology Advancement in Engineering*, *2*(12), 43-52.
8. Putra, G. R. (2025). Computer Programming with JAVA Java JumpStart: A Hands-on Training from Basics to Building Blocks.
9. Kumar, B. P., Tarun, K., & Ganesh, P. V. P. (2025). A smart and efficient college management system using full-stack technologies. *Journal of Nonlinear Analysis and Optimization*, *16*(1).
10. Zhao, Z. (2025). Design and implementation of a college student comprehensive quality evaluation system based on Spring Boot.

Appendices

The appendices contain the **supporting information and materials** used during the development of the project. They include sample student academic data, details of the two

modules, sample Java code, performance calculation, attendance alerts, sample system output, testing results, and possible future improvements. These materials provide additional information about how the **Student Academic Management System** was designed, implemented, and tested. The appendices provide additional details related to the project that support the main report. They include **sample student records, module details, Java program code, performance classification criteria, attendance alert examples, testing results, and sample system outputs**. These materials help demonstrate how the proposed system works and provide supporting evidence for the design and implementation of the **Student Academic Management System**.

CODE

```
Import java.util.Scanner
```

```
class Student {  
  
    int studentId;  
  
    String name;  
  
    double attendance;  
  
    double assignmentScore;  
  
    double quizScore;  
  
    double internalMarks;  
  
    double studyHours;  
  
    Student(int studentId, String name, double attendance,  
            double assignmentScore, double quizScore,  
            double internalMarks, double studyHours) {  
  
        this.studentId = studentId;  
  
        this.name = name;  
  
        this.attendance = attendance;  
  
        this.assignmentScore = assignmentScore;  
  
        this.quizScore = quizScore;  
    }  
}
```

```

        this.internalMarks = internalMarks;

        this.studyHours = studyHours;
    }

    void displayStudentDetails() {

        System.out.println("Student ID    : " + studentId);

        System.out.println("Student Name  : " + name);

        System.out.println("Attendance   : " + attendance + "%");

        System.out.println("Assignment Score : " + assignmentScore);

        System.out.println("Quiz Score    : " + quizScore);

        System.out.println("Internal Marks  : " + internalMarks);

        System.out.println("Study Hours    : " + studyHours);

    }

    double calculatePerformance() {

        return (assignmentScore + quizScore + internalMarks) / 3;

    }

    String getPerformanceCategory() {

        double score = calculatePerformance();

        if (score >= 85)

            return "Excellent";

        else if (score >= 70)

            return "Good";

        else if (score >= 50)

            return "Average";
    }

```

```

        else

            return "Poor";

    }

    void checkAttendance() {

        if (attendance < 80)

            System.out.println("Attendance Alert : YES - Faculty Attention Required");

        else

            System.out.println("Attendance Alert : NO");

    }

}

public class Main {

    public static void main(String[] args) {

        Student s1 = new Student(

            101, "Arun", 92, 85, 88, 90, 3

        );

        Student s2 = new Student(

            102, "Priya", 86, 78, 82, 80, 2

        );

        Student s3 = new Student(

            103, "Rahul", 76, 68, 72, 70, 2

        );

        Student s4 = new Student(

            104, "Meena", 65, 55, 60, 58, 1

        );

```

```

Student s5 = new Student(
    105, "Karthik", 94, 92, 95, 91, 4
);

System.out.println("=====");

System.out.println(" STUDENT ACADEMIC MANAGEMENT SYSTEM");

System.out.println("=====");

s1.displayStudentDetails();

System.out.println("\n-----");

s2.displayStudentDetails();

System.out.println("\n-----");

s3.displayStudentDetails();

System.out.println("\n-----");

s4.displayStudentDetails();

System.out.println("\n-----");

s5.displayStudentDetails();

System.out.println("\n\n=====");

System.out.println(" PERFORMANCE MONITORING AND DECISION SUPPORT");

System.out.println("=====");

displayPerformance(s1);

displayPerformance(s2);

displayPerformance(s3);

displayPerformance(s4);

displayPerformance(s5);

}

```

```
static void displayPerformance(Student s) {  
  
    double performance = s.calculatePerformance();  
  
    System.out.println("\n-----");  
  
    System.out.println("Student ID : " + s.studentId);  
  
    System.out.println("Name      : " + s.name);  
  
    System.out.println("Performance : " + performance);  
  
    System.out.println("Category   : " + s.getPerformanceCategory());  
  
    s.checkAttendance();  
  
}  
  
}
```